Software Development Life Cycle (SDLC) (Agile focused)

Key Takeaways

Flexible Framework: The SDLC is a versatile framework that unites various methodologies like Agile, Scrum, and DevOps, structuring the software development process from inception to launch.
Standardization and Safeguarding: Even with AI's rise, the SDLC is crucial for maintaining structured processes. It helps integrate AI tools into workflows efficiently, avoiding disorder and enhancing productivity.
Who Does What: The SDLC clarifies roles, facilitates better planning, and supports testing and iteration. This helps product teams stay coordinated, minimizing waste and delivering reliably.

Other Lifecycle Management Methodologies

- Application Lifecycle Management
- Systems Development Lifecycle
- Software Testing Lifecycle
- DevOps
- Product Development Lifecycle
- Software Release Life Cycle

Common SDLC Models

1. **Agile Model**
This model arranges the SDLC phases into several development cycles, with the team delivering small, incremental software changes in each cycle. The Agile methodology is highly efficient, and rapid development cycles help teams identify issues early on, but overreliance on customer feedback could lead to excessive scope changes or project termination. It's **best for software development projects that require flexibility** and the ability to adapt to change over time.

2. **Waterfall Model**
This model arranges all the phases sequentially, with each new phase depending on the outcome of the previous one. It provides structure to project management, but there is little room for changes once a phase is complete, so it's **best for small, well-defined projects**.

3. **Iterative Model**
With this model, the team begins development with a small set of requirements and iteratively enhances versions until the software is ready for production. It's easy to manage risks, but repeated cycles could lead to scope change and underestimation of resources. This model is **best for projects that require high flexibility in their requirements** and have the resources to handle multiple iterations.

4. **Spiral Model**
This model combines the iterative model's repeated cycles with the waterfall model's linear flow to prioritize risk analysis. It's **best for complex projects** with frequent changes but can be expensive for smaller projects.

5. **Big Bang Model**
The Big Bang Model is a unique approach where developers jump right into coding without much planning. This means that requirements are implemented as they come, without any kind of clear roadmap. If changes are needed, it can require a complete revamp of the software. While this model isn't great for larger projects, it's **best for academic or practice projects, or smaller projects with only one or two developers**. Essentially, it's a model that works well when requirements aren't well understood and there's no set release date in sight.

Weeks numbers are set as an example. They vary strongly from project to project.

With this color PM Related Activities are marked

4-5 days per week **Daily** meeting takes place every phase

Other **rituals** are included in weekly split of activities as it doesn't make sense to separate them - this will duplicate info in many cases

Glossary

Agile & Development Process Terms
- **Backlog**: A prioritized list of work items (epics, user stories, tasks) that the team may work on in the future.
- **Refinement I**: PM-led session to share product context, walk through epics/user stories, and gather initial technical/UX feedback.
- **Refinement II**: Dev-led session focused on breaking stories into tasks, discussing technical implementation, and estimating effort.
- **Sprint Planning**: A meeting where the team decides which backlog items to commit to for the upcoming sprint (typically 1–2 weeks).
- **Sprint**: A time-boxed development cycle in Agile, typically 1–4 weeks, during which the team delivers a working increment.
- **Story Points**: A relative unit of effort/complexity used to estimate how much work a user story will require.
- **Feature Flag**: A technique to release code that is not yet visible or enabled for all users, allowing gradual rollout or internal testing

Artifacts
- **Epic**: A large body of work that can be broken into smaller user stories. Represents a key functionality or feature set.
- **User Story**: A short, simple description of a feature told from the perspective of the user (e.g., "As a manager, I want to…").
- **Acceptance Criteria**: Conditions that must be met for a story to be considered complete and ready for release.
- **Definition of Done**: A team's shared agreement on what must be true for any item to be considered "done" (e.g., tested, documented, reviewed).
- **Interface Contract**: A documented agreement between backend/frontend (or between services) describing expected data structures and behavior.
- **Fallback State**: What the user sees when something fails (e.g., AI output is missing, API doesn't respond).

Technical & AI Concepts
- **Prompt**: A structured input sent to a language model (LLM) to generate a response. Can be templated and versioned.
- **Few-shot prompting**: Providing a few examples in a prompt to guide the AI's behavior.
- **Fine-tuning**: Training a model on domain-specific data to improve its output quality for a particular use case.
- **Confidence Threshold**: A minimum confidence score below which an AI model's output is considered unreliable and a fallback should be shown.
- **LLM (Large Language Model)**: An AI system trained on massive text data to generate human-like responses.
- **Prompt Lifecycle Tracker**: A document tracking versions, owners, improvements, and known issues of AI prompts.
- **Model Health Contract**: A defined set of rules about AI output quality, error handling, confidence scoring, and escalation processes.
- **Guardrails**: Constraints placed on AI systems to ensure safe, reliable, or on-brand output.

Testing Types
- **Unit Testing**: Testing individual units of code in isolation (e.g., a single function or button).
- **Integration Testing**: Ensuring different parts of the system (e.g., backend and frontend, or backend and AI) communicate correctly.
- **End-to-End Testing (E2E)**: Testing the full workflow from user input to system response and final output.
- **Regression Testing**: Re-running old test cases to ensure new changes don't break existing functionality.
- **Load & Performance Testing**: Checking system response time and behavior under high usage conditions.
- **Smoke Testing**: A lightweight check to verify that the core functionality works in a live or staging environment.
- **Usability Testing / X-Ray Testing**: Observing users interacting with the product to identify usability issues and drop-off points.
- **A/B Testing**: Comparing two variants (A and B) to determine which performs better with users.

Roles
- **PM (Product Manager)**: Defines the what and why of the product. Coordinates between stakeholders, users, and tech.
- **Tech Lead**: Senior engineer responsible for architectural decisions, tech roadmap, and resolving blockers.
- **QA (Quality Assurance)**: Ensures product meets quality standards through testing and validation.
- **Security Champion**: Engineer or team responsible for identifying and addressing security risks.
- **Prompt Engineer**: Person who crafts and iterates on AI prompts to ensure good model behavior.
- **Designer (UX/UI)**: Responsible for designing user interfaces and flows that are intuitive and meet user needs.

Delivery & Tracking
- **Iteration Board**: A tracking board (e.g., Jira, Airtable) for organizing and prioritizing ongoing fixes and improvements.
- **Golden Set**: A standard set of input/output examples used to test and evaluate model performance over time.
- **Release Notes**: Documentation of what is included in a release, caveats, and usage guidance.
- **UAT (User Acceptance Testing)**: Final validation by users or business stakeholders to confirm a product is ready for release.

Phase 1: Planning

Inputs to the Phase

- **Strategic Objectives** Business vision, OKRs, company goals the product is expected to support.
- **Problem Statement** Clear articulation of the user or business problem the product should solve.
- **Initial Stakeholder Map** List of internal/external stakeholders (e.g., legal, tech, support, marketing, execs).
- **Market Landscape Overview** Competitive analysis, market trends, and benchmarks relevant to the domain.
- **Initial Personas or Customer Segments** Draft personas representing potential end users (can be updated in design phase).
- **Assumptions & Constraints** Known limitations (e.g., timeline, budget, security, platforms, data residency).
- **High-Level User Outcomes** Expected improvements in user behavior, productivity, or satisfaction.
- **Regulatory/ Compliance Signals** Early awareness of legal boundaries (e.g., GDPR, financial disclosures).
- **Preliminary Tech Landscape Scan** Current systems in place, relevant platforms or APIs, internal tooling available.

Definition: The planning phase is the foundation of any successful software development project. Project goals, objectives, and requirements are gathered and documented during this phase. Project requirements can be based on customer feedback or market research evaluating existing product options. Stakeholders work together to define the project scope, establish timelines, and allocate resources. Planning establishes the project's direction, ensuring that all participants have a clear understanding of what needs to be done and how to achieve it.

Week 1 | Week 2 | Week 3
- Kick-off Alignment Workshop
- Stakeholder Discovery Interviews
- Persona & Journey Mapping Session
- Business Case Review
- Vision & Success Metrics Workshop
- Initial Roadmap Alignment
- Planning Phase Sign-off Review

Outputs of the Phase

- **Product Vision Statement** A concise articulation of what the product will achieve and for whom.
- **Stakeholder Alignment Notes** Agreements on scope, expectations, and roles (can be in Notion, Confluence, or shared decks).
- **Persona Profiles (Lightweight)** Initial representations of key user types or audiences.
- **Draft Product Roadmap** High-level timeline and scope for major releases or MVP.
- **Assumptions Log** Written record of assumptions to be validated later (technical, legal, market).
- **Risks & Constraints Register** Documented legal, infra, or timeline constraints to consider downstream.
- **Problem Definition Document** Clear documentation of the core user or business problem to solve.
- **Preliminary Success Metrics (North Star)** Draft indicators for what success looks like (e.g., adoption %, time saved).

Common Challenges & Mitigations for the Phase

- **Misaligned Stakeholder Expectations** **Mitigation:** Run alignment workshops early to define shared success criteria and clarify what's in/out of scope.
- **Vague or Overly Ambitious Vision** **Mitigation:** Translate product vision into concrete problem statements and core use cases; validate them with user interviews.
- **Resource Uncertainty** **Mitigation:** Create multiple planning scenarios (e.g., MVP vs. full scope); define resource assumptions for each.
- **Prioritization Chaos** **Mitigation:** Use prioritization frameworks (e.g. RICE, MoSCoW) and link backlog to business impact, not just stakeholder influence.

PM Activities	Activities Output
Identify and define business problem / opportunity	Problem definition document or strategic brief
Align with stakeholders on goals and priorities	Stakeholder alignment matrix or kickoff notes
Conduct initial market/user research	Research summary or competitor analysis
Map high-level user needs and workflows	Early persona profiles and user journeys
Define product vision and success metrics	Vision statement, OKRs, or product KPIs
Outline strategic options and scope boundaries	Draft product scope and constraints doc
Evaluate internal capabilities and timelines	Resource plan / team capability assessment
Plan for cross-functional team setup	Roles & responsibilities matrix
Create high-level roadmap or delivery phasing	Draft release roadmap (quarterly or milestone-based)

Core Planning Phase Team

- **Product Manager** (drives product vision, coordinates stakeholder input, defines early roadmap)
- **Business Lead / Sponsor** (sets business goals, budget, and strategic priorities)
- **UX Researcher / Product Designer** (conducts early discovery interviews, helps define problem space)
- **Engineering Lead / Architect** (assesses technical feasibility, flags architectural constraints)
- **Data Analyst** (brings insight from early user/data research if available)
- **Legal or Compliance Advisor (optional)** (flags early regulatory constraints, if AI/data is involved)

Phase 2: Feasibility analysis

Planning & Feasibility analysis phases can intercross

Inputs to the Phase

- **Existing System Architecture Diagrams** To assess integration complexity and infrastructure alignment.
- **Data Source Inventory** What data exists, where it lives, who owns it, and what the quality is like.
- **Access to Technical SMEs** Engineers, data scientists, or architects who can evaluate constraints and unknowns.
- **Preliminary Legal and Compliance Checklist** Initial review of privacy, security, and data-sharing implications.
- **Sample Data Sets or API Schemas** To test edge cases, feasibility of rule-based or AI-based decisions.
- **Risk/Complexity Assumptions Log** Identified uncertainties and critical blockers that could delay or prevent execution.
- **Resource Availability Plan** Signals of team bandwidth or required hiring needs.
- **Proof of Concept Prototypes (Optional)** Low-fidelity models or technical spikes that validate risky assumptions.

Definition: Once planning is complete, the feasibility analysis phase begins. During this phase, the project team evaluates whether the project is technically and financially viable. This includes assessing the technical requirements, estimating costs, and performing a risk analysis. Risk assessment is essential to identifying potential challenges and determining if the project is worth pursuing.

Week 1 | Week 2 | Week 3
- Technical Feasibility Deep Dive
- Compliance Feasibility Workshop
- Data Source Alignment Session
- Prototype or Spike Review
- Buy vs Build Assessment Checkpoint
- Cross-Functional Risk Assessment
- Feasibility Gate Review

Outputs of the Phase

- **Feasibility Report** Assessment of legal, technical, data, and organizational feasibility.
- **Tech & Data Architecture Snapshot** Sketch of integration points, data flows, and backend/infra needs.
- **Data Availability Analysis** Findings on what data exists, its structure, quality, and gaps.
- **Legal & Compliance Risk Assessment** Initial go/no-go areas and regulatory boundaries.
- **List of Integration or API Dependencies** Documented external or internal systems that must be connected.
- **Validation Results** Outcomes from spike/proof-of-concept (PoC) or sandbox testing.
- **Updated Product Scope** Refined MVP or pilot definition based on constraints discovered.
- **Go/No-Go Recommendation** Formal decision whether to proceed, pivot, or delay development.

Common Challenges & Mitigations for the Phase

- **Technical Unknowns / AI Readiness Gaps** **Mitigation:** Run technical spikes to explore architectural and AI model feasibility early; document API/data assumptions.
- **Poor Data Quality or Access** **Mitigation:** Create a data audit checklist; identify minimum viable sample; work with data owners to unblock access.
- **Legal Review Bottlenecks** **Mitigation:** Shift legal to async reviews with batch cycles; prioritize flagged cases first.
- **Overengineering Risks** **Mitigation:** Define what "good enough" looks like for each component; frame V1 as a learning platform.

PM Activities	Activities Output
Facilitate technical feasibility workshops with Engineering	Feasibility matrix (high/ medium/low) for key features
Assess legal, compliance, and security constraints	Risk register or feasibility summary from legal/security
Coordinate with Data/AI/IT teams on data availability and privacy	Data feasibility assessment
Explore buy vs. build options or partner integrations	Tech options comparison table
Validate assumptions with a spike, POC, or prototype	Prototype test results or POC demo feedback
Conduct ROI and cost-benefit analysis	Business case or lean canvas
Decide go/no-go or pivot strategy with stakeholders	Decision memo or pivot strategy document

Core Feasibility Analysis Phase Team

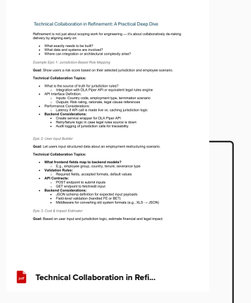
- **Product Manager** (frames core problem, defines success metrics, owns discovery outcomes)
- **Technical Lead / Solution Architect** (assesses architectural implications, integration needs)
- **Data/ML Engineer** (evaluates availability and structure of training/ operational data)
- **Legal SME** (assesses compliance feasibility for proposed solution)
- **Security Lead (if required)** (validates early assumptions for secure integrations)
- **Customer / Domain SME** (validates realism of proposed solution from user and business lens)

Phase 3: Design

Inputs to the Phase

- **Product Vision** clear articulation of the product's goals and user value.
- **Problem Statement** what challenge are we solving and for whom.
- **Target Personas / Segments** user types, roles, motivations, and behaviors.
- **High-Level Use Cases** key scenarios to support (e.g., "HR manager models workforce scenario").
- **High-Level Use Cases** key scenarios to support (e.g., "HR manager models workforce scenario").
- **Requirements Backlog (initial)** descriptions of user goals (early user stories or jobs-to-be-done).
- **Process Diagrams / Workflows** visual breakdown of how users move through the product or service.
- **System Constraints** what the system can or cannot do due to regulatory, technical, or architectural limitations.
- **Technical Feasibility Notes** confirmed scope boundaries, known risks, and system constraints.
- **Integration Considerations** APIs or data sources to be used, any dependencies already identified.
- **Initial Architecture Direction** guidance from Tech leads or architects on systems, data flows, security.
- **Performance Constraints or SLAs** required loading speeds, model latency, data refresh cycles.
- **Compliance Requirements** privacy, auditability, AI explainability, jurisdictional nuances.
- **Data Governance Rules** what kind of data can be stored, how long, who accesses it.
- **Design System** Figma components, color palette, typography, UI patterns.
- **Brand Guidelines** tone, style, visual identity.
- **Previous Product Components or Screens** reuse of previous UI, APIs, models, or flows if applicable.
- **MVP Scope Agreement** rough scope, success criteria, and out-of-scope items.
- **Timeline / Roadmap Commitments** launch target dates, dependencies (e.g., pilot dates).
- **Stakeholder Expectations** especially executive, sales, legal, or marketing.

Definition: The system design phase includes creating the software's architecture and design. Based on the requirements gathered during planning, the team creates a blueprint outlining how the software will function. This includes high-level architecture and detailed design specifications, including user interface design to ensure the software is user-friendly and an assessment of requirements for compatibility with existing products.



Technical Collaboration in Refi...

Week 1 | Week 2 | Week 3 | Week 4
- Discovery Kick-off & Scope Alignment
- Stakeholder Interviews
- User Journey Mapping
- Functional Requirements Draft
- Wireframe Sketching (Lo-Fi)
- Design Feedback Loop #1
- High-Fidelity Prototypes
- API Contract Discussions
- Design Feedback Loop #2
- Define Edge Cases & Error States
- Storyboarding for Core Scenarios
- Final Design Sign-Off
- Backlog Grooming (Refinement): Epics & Stories
- Sprint Planning Prep
- Documentation Publishing

Optional Light Estimation of Story Points for User Stories

Refinement I as the bridge to next Phase

Refinement II as the input for Sprint planning
Activities:
- Decomposition of User Stories into technical tasks.
- Identification of dependencies, blockers, unknowns.
- Estimation of effort using story points (e.g., using Fibonacci scale: 1, 2, 3, 5, 8...).

Estimation of Story Points for Backlog Items

Outputs of the Phase

- **Low- and High-Fidelity Wireframes** Sketches and finalized UI layouts representing user journey and screens.
- **Clickable Prototype** Interactive design prototype for testing user flow and usability (e.g., in Figma).
- **Information Architecture** Navigation hierarchy and layout of content and interaction patterns.
- **UI Design System or Component Library** Shared visual elements for consistency (buttons, inputs, modals).
- **Technical Design Documents** Diagrams and documentation for frontend/backend architecture, API contracts, schema.
- **System Sequence Diagrams or Flowcharts** Logic of system interactions, LLM flows, integration points.
- **Prompt Design Specs (for AI products)** Initial prompts, versioning logic, fallback design, confidence threshold rules.
- **Security & Privacy Requirements** Defined data handling, encryption, user access rules.
- **Initial Acceptance Criteria per Epic/ User Story** Clarity on what "done" means for each functional unit.
- **Validated User Journey Maps** Persona-based maps showing pain points and interactions across the experience.
- **Feedback Log from Stakeholders** Annotated feedback from legal, marketing, or internal SMEs on usability and risk.

Common Challenges & Mitigations for the Phase

- **Scope Creep During Wireframing** **Mitigation:** Lock MVP functionality with user stories before detailed UI work; trace every screen to a specific user need.
- **Lack of Developer Input in Design** **Mitigation:** Involve engineers early in design reviews; use design–tech pairing sessions for feasibility feedback.
- **Ignoring Edge Cases** **Mitigation:** Create an edge case playbook per screen or flow; define fallback behavior (e.g., null, malformed, unsupported inputs).
- **Unclear Handoff Between Design and Dev** **Mitigation:** Use shared tools (e.g., Figma with annotations, Jira integration); document states, interactions, and validations.

PM Activities	Activities Output
Clarify product vision and objectives	Vision summary, aligned problem statement
Conduct stakeholder alignment sessions	Prioritized list of business needs and success criteria
Define MVP scope	MVP scope document with clear in-scope and out-of-scope features
Coordinate with UX to shape user journeys	Validated user flows, UX personas, journey maps
Break product scope into functional modules (early epics)	Initial epic map or capability breakdown
Facilitate system mapping with tech leads	Preliminary architecture diagram / data flow chart
Initiate legal and compliance consultations	Documented design constraints (e.g., access, audit, jurisdiction)
Drive early usability hypothesis and assumptions	Usability assumptions checklist, annotated mockups
Create and review wireframes/ prototypes	Figma files with PM-reviewed comments and annotations
Collect feedback from legal, stakeholders, SMEs	Consolidated design feedback log
Document edge cases and fallback behaviors	Edge case inventory (including error handling notes)
Ensure integration requirements are covered	Integration readiness checklist (API/ data source notes)
Write high-level user stories with technical input	Refined user stories ready for Refinement I
Prepare for Refinement I session	Presentation deck with context, personas, functionality, and questions

Use prioritization frameworks (RICE, MoSCoW, etc.)

Core Design Phase Team

- **Product Manager** (aligns user stories to designs, tracks MVP scope, manages decisions)
- **UX/UI Designer** (leads interaction, screen design, clickable prototypes)
- **Engineering Lead (Frontend/ Backend)** (validates technical feasibility of UI/UX flows)
- **Legal SME (if required)** (reviews copy, UX elements that imply legal or regulatory meaning)
- **AI/ML Engineer (if AI output shown in UI)** (confirms model interaction points)
- **QA Lead (optional)** (defines early validation scenarios based on flows)

Phase 4: Implementation/Build/Devlopment

Inputs to the Phase

- Approved epics and user stories with clear acceptance criteria
- Designs/ wireframes and UX flows
- Prioritization from PM/PO (sprint backlog)
- Estimates from engineering team
- Defined architecture and dependencies

Inputs to the Phase (AI Product)

- Clean, labeled training datasets (or access to them)
- Evaluation metrics and baseline benchmarks
- Model architecture decisions or exploratory prototypes

Definition: The implementation phase, also known as the development phase, transforms the design into a functional application. It is here that the actual coding takes place. Developers write the code based on the design specifications, following best practices and coding standards to ensure the result is efficient, secure, and maintainable.

Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 | Week 8
- Sprint 1 Planning
- Backend Setup
- Frontend Skeleton
- Prompt Engineering / AI Integration Start
- Sprint 1 Review + Retrospective
- QA Test Plan Setup
- Sprint 2: UI Components + Model Piping
- Sprint 2 Review + Stakeholder Demo
- Update Stories Based on Feedback
- Sprint 3: Complex Flows, Error States
- Continue Prompt Refinement & Logging
- Sprint 3 Review + UI/ Legal Sign-Off
- Sprint 4: Performance + Security Enhancements
- Sprint 4 Review + Go/No-Go Decision Prep
- UAT with Legal + Pilot Stakeholders
- Final Release Note + Enablement Materials
- Go/No-Go Meeting

Outputs of the Phase

- **Working Product Features (Incremental Releases)** Built and tested vertical slices that include backend logic, UI, and API connections.
- **Updated Source Code in Repository (with Versioning)** CI/CD-integrated codebase with branching, commits, and documentation.
- **API Documentation** Interface contracts, endpoints, schemas, and authentication flows.
- **Integration Logs** Backend-frontend integration notes, system tests, and external service mappings.
- **Prompt Version Control Log** Record of prompt changes, test results, and rollback instructions (AI-specific).
- **Test Artifacts for Unit/Integration Testing** Coverage reports, automated test suites, and testing documentation.
- **Feature Flags Configuration** Documentation on enabled/disabled features in staging vs. prod.
- **Developer Notes on Known Limitations** Handoff notes for QA or legal on areas not covered, hardcoded assumptions.
- **Dev Demo Presentations** Internal walk-throughs to align QA, PM, legal before testing phase.

Common Dependencies & Mitigations for the Phase

- **Technical Dependencies** Dependencies related to other components, services, or teams delivering technical assets or APIs your product relies on.
  Mitigations:
  - Define interface contracts early in refinement (OpenAPI specs, data contracts)
  - Use mock services or dummy endpoints to unblock frontend development
  - Establish sync points between teams (e.g., alignment demos or standups)
- **Infrastructure Dependencies** Constraints or reliance on internal infrastructure, such as authentication, deployment, or hosting environments.
  Mitigations:
  - Involve platform teams during refinement to assess feasibility
  - Use layered architecture so product logic can evolve independently
  - Timebox MVP scope to work with current infrastructure, defer advanced controls for later
- **Sequencing Dependencies** Tasks or modules must happen in a certain order due to logic or data flow.
  Mitigations:
  - Use dependency mapping in Jira (e.g., linked issues, blockers)
  - Plan minimum sequence in refinement; keep downstream teams in sync
  - Leverage feature flags to deploy partially built features without breaking flows
- **Compliance or Legal Dependencies** Certain logic or flows require sign-off or direction from legal/ compliance stakeholders.
  Mitigations:
  - Use dependency mapping in Jira (e.g., linked issues, blockers)
  - Plan minimum sequence in refinement; keep downstream teams in sync
  - Leverage feature flags to deploy partially built features without breaking flows
- **AI/ML Dependencies** When product modules rely on evolving or non-deterministic ML behavior.
  Mitigations:
  - Version control for prompts
  - Logging all outputs
  - Add fallback UIs when confidence is low

PM Activities	Activities Output
Backlog Management	Stories are groomed and prioritized
Engineering Support	Clarify feature logic and edge cases
Unblock & Align	Coordinate dependencies
Sprint Rituals	Facilitate and track progress
Define test scope and goals for UAT & QA	Test scope document, aligned acceptance criteria
Support test case creation with QA / engineers	PM-reviewed test case list tied to user stories
Run smoke tests / demo validations pre-UAT	Pre-UAT issue log or go/no-go checklist
Facilitate UAT with legal, business, or pilot users	UAT feedback log, signed acceptance or rework tickets
Review and triage test failures	Triage board with severity, owner, and resolution plan
Validate that edge cases and fallback logic work as expected	Verified test results covering edge/ failure scenarios
Check compliance with explainability and legal standards	Legal validation checklist
Coordinate fix cycles and retesting rounds	Retest tracker (e.g., spreadsheet or JIRA board)
Document known limitations or warnings for release notes	"Known Issues" section in release note draft
Prepare training materials or enablement guides for users	Final 1-pager, walkthrough videos, FAQs
Confirm monitoring/ logging is in place post-release	Monitoring plan and alerts checklist
Conduct post-test review with dev and QA	Retrospective notes and quality learnings

Core Builde Phase Team

- **Product Manager** (owns user stories, feedback loop, testing prep, daily alignment)
- **Frontend / Backend Engineers** (implement vertical slices from UI to DB/API layer)
- **AI/ML Engineer** (integrates model API, manages prompt structure, tests edge cases)
- **QA Engineer** (writes and runs automated/manual tests, manages smoke and regression tests)
- **Security Champion** (validates compliance with security practices)
- **Designer** (supports dev-ready screens and iterations based on dev feedback)
- **Legal / Compliance (optional)** (involved in sprint reviews, especially with AI output)

Phase 5: Testing

Testing phase intercrossing with Build phase

Definition: The testing phase is critical because it generates essential performance and usability feedback while revealing defects and quirks. Various types of software testing can be used, including automated testing, unit testing, integration testing, and system testing. The goal is to identify and fix bugs, ensuring the software operates as intended before being deployed to users.

- Unit Testing
- Integration Testing
- API Contract Testing
- Frontend Smoke Testing
- Regression Testing
- End-to-End (E2E) Testing
- Load & Performance Tests

Outputs of the Phase

- **Unit Test Results** Pass/fail logs per function or component.
- **Integration Test Results** Validation of correct data exchange between modules (e.g., backend ↔ LLM ↔ UI).
- **End-to-End (E2E) Test Scenarios and Logs** Confirmation that entire user flow behaves as expected (e.g., via Cypress, Selenium).
- **Regression Test Reports** Confirmation that newly added features didn't break previous functionality.
- **UAT Sign-Off Sheets** Internal (PM, legal, design) or client sign-off for release-readiness.
- **Bug Reports & Issue Tracker Export** Prioritized list of bugs, their severity, owners, and fixes.
- **Performance and Load Testing Results** System behavior under normal and peak load, response times, bottlenecks.
- **Security & Penetration Test Reports (if applicable)** Compliance documentation for data handling and user privacy.
- **Fallback Scenario Logs** Edge case and failure mode behavior for AI systems, with screenshots or logs.
- **QA-to-PM Summary Report** Coverage, blocker status, and release readiness from QA lead.

Phase 6: Deployment/Release

Inputs to the Phase

- **Final Tested Build** The production-ready version of the software, ideally with feature flags.
- **UAT (User Acceptance Testing) Sign-off Artifacts** Evidence that legal, business, and QA users have validated the product's readiness.
- **Security & Compliance Approvals** Sign-off on data handling, access control, encryption, legal disclaimers, and retention policies.
- **Monitoring & Logging Configuration** Alert systems and logs prepared to observe the system post-release.
- **Deployment Runbook** Step-by-step process for releasing the product (e.g., environments, rollback paths).
- **Enablement Materials** Internal slide decks, FAQs, demo videos, escalation flowcharts.
- **Support Playbooks** Documentation for support teams on how to resolve potential issues or triage bugs.
- **Release Communication Plan** Stakeholder-facing email templates, announcement plans, changelogs.
- **Legal Disclaimers & Terms of Use** Final approved copy to appear on the platform, if AI or risk-based outputs are involved.

Definition: Once internal software testing is complete, the solution can be deployed to end users. This typically includes a beta-testing phase or pilot launch, limited to a select group of real-world users. Depending on the project's needs, deployment can be done on-premise or in the cloud. The deployment strategy determines how easily users can access and use the software.

Week 1 | Week 2
- Enablement Dry Run
- Security & Legal Compliance Checkpoint
- Production Environment Readiness Review
- Internal Training Session
- Controlled Pilot Launch Monitoring
- Release Retrospective

Outputs of the Phase

- **Live Product in Production** Codebase deployed and active for pilot users or general access.
- **Release Notes** Document describing what's included, caveats, and disclaimers.
- **UAT Sign-off Sheet** Legal, business, and QA stakeholders confirm readiness.
- **Monitoring Dashboards** Live observability tools for system health, errors, and usage metrics.
- **Internal Enablement Assets** Quick-start guides, walkthrough videos, feature overviews for internal teams.
- **Support Escalation Playbook** Clear flow of how issues are reported, triaged, and resolved.
- **Security and Compliance Log** Audit trail of sign-offs, encryption setup, access restrictions.
- **Training & Demo Materials** Slides and demo scripts for internal champions or client onboarding.
- **User Communication** Email, in-app banners, or release posts targeting end-users.

Common Challenges & Mitigations for the Phase

- **Legal or Compliance Delays** **Mitigation:** Engage legal async 2–3 sprints prior to release; maintain a backlog of fallback disclaimers and approved UI copy.
- **Unstable Model Outputs or Hallucinations** **Mitigation:** Validate outputs on golden dataset during UAT; configure frontend to trigger fallback states if AI confidence thresholds are not met.
- **Missing Post-Release Monitoring** **Mitigation:** Build dashboards ahead of go-live; assign owners for daily post-release health checks.
- **Last-Minute Functional or AI Bugs** **Mitigation:** Establish pre-defined rollback strategy; stage rollback rehearsals.
- **Stakeholder Misalignment on Scope** **Mitigation:** Use centralized Release Readiness Doc with version-controlled signoffs from each function.

PM Activities	Activities Output
Define release scope and version boundaries	Final release note or version matrix
Prepare internal documentation and enablement	Product enablement deck, 1-pager, FAQs
Set up customer support materials and escalation paths	Support playbook or escalation flow
Coordinate with Legal, Security, and Compliance	Final approvals or sign-offs (e.g., T&Cs, audit trail)
Manage feature flags and environment rollouts	Feature flag status doc, environment checklist
Lead internal launch training / comms	Training schedule, live demo session materials
Deploy to pilot/internal/ external users as needed	Launch status report
Track adoption metrics and post-release performance	Post-launch dashboard (engagement, issues)

Core Release Team

- Product Manager (owns full release plan and coordination)
- Backend Engineer / Frontend Engineer (deployment readiness)
- DevOps / Infrastructure Engineer (pipeline, feature flags, rollback)
- QA (final testing, regression, smoke tests)
- Security Champion (access control, audit trail validation)
- Legal SME (regulatory sign-offs, disclaimers)
- Marketing (enablement and launch communications)

Phase 7: Maintenance

Definition: Even after the software is deployed, ongoing support is necessary to address issues, apply updates, and add new features. Continuous maintenance ensures that the software remains functional and relevant over time.

Frequency	Ritual	Owner	Purpose
Weekly	Bug Triage Stand-up	PM, QA, Dev	Review open bugs, classify severity, assign for fix
Bi-weekly	Live Ops Sync	PM, Support, Engineering	Address major incidents, user complaints, monitoring alerts
Monthly	Product Health Review	PM, Data Analyst	Analyze usage metrics, drop-offs, regression issues
Monthly	Compliance Revalidation Ritual	PM, Legal, Security	Confirm no infra/legal regressions; rotate credentials; audit data access
Quarterly	Deprecation Planning Checkpoint	PM, Engineering	Identify legacy features to sunset, create migration paths
Ad hoc	Hotfix Release Sync	PM, QA, Engineering	Prepare urgent release flow and communication for high-severity issues
Ad hoc	Feedback Processing Review	PM	Process user suggestions, update backlog accordingly

Common Challenges & Mitigations for the Phase

- **Iteration Work Losing Priority to New Features** **Mitigation:** Reserve fixed capacity (% of every sprint) for stability work. Tie iteration goals to OKRs.
- **Lack of Labeled Failure Data** **Mitigation:** Build simple "flag output" buttons for legal users and support staff to submit failure examples.
- **Legal Review Bottlenecks** **Mitigation:** Shift legal to async reviews with batch cycles; prioritize flagged cases first.
- **AI Model Changes Causing Unexpected Regressions** **Mitigation:** Use golden set test suites to validate model changes before deployment.
- **Misalignment on Success Criteria** **Mitigation:** Define iteration metrics upfront (e.g., fallback reduction, override rates, user NPS scores on trust).

PM Activities	Activities Output
Monitor product health and user behavior	Live dashboards, alerts, and incident logs
Triage incoming bugs and user issues	Bug backlog or triage board
Coordinate with Support/ Engineering on fixes	Hotfix plan or weekly fix cycles
Collect ongoing user feedback	Feedback repository or Net Promoter Score (NPS) tracking
Evaluate performance against KPIs	Monthly KPI review report
Prioritize iterative improvements or enhancements	Improved backlog
Plan minor or patch releases	Patch release scope
Revisit security and compliance needs	Recertification checklist or compliance review
Drive deprecation planning for legacy features	Sunset plan or migration roadmap
Enable customer education and updates	Changelog updates, new feature highlights

Core Iterate Team

- Product Manager (leads iteration planning, backlog grooming, feedback synthesis)
- Backend / Frontend Engineers (implement iterative changes)
- AI/ML Engineer (model monitoring, prompt tuning, failure analysis)
- Designer (if UI adjustments are involved)
- Legal SME (reviews any AI or compliance-affecting changes)
- Customer Success / Support (frontline feedback aggregation)

Edge Cases

Edge Case 1: Missing or Incomplete Input Data
Epic Context: Scenario Input CaptureDescription: A user submits the input form but omits key fields, or submits poorly formatted data (e.g., no headcount, invalid jurisdiction code, or unsupported characters in free-text fields).
Solution:
- Display clear, inline validation messages (e.g., "Please enter a valid headcount" or "This jurisdiction is not currently supported.")
- Disable submission button until required fields are filled and correctly formatted
- For unsupported jurisdictions, provide a non-blocking fallback panel with:"We don't yet support automated results for this jurisdiction. You can proceed, and our legal team will review this manually."
- Log all invalid submissions (including timestamps and user IDs if permitted under compliance rules) to improve system training or coverage.

Edge Case 2: LLM Fails to Return Valid Output
Epic Context: LLM Integration & Response EngineDescription: The LLM API (e.g., OpenAI or internal model) fails to return a result — due to timeout, malformed input, or model failure — or returns something unusable (e.g., hallucinated text or incomplete structure).
Solution:
- Implement backend fallback logic:If the model response is empty or fails schema validation → return {"status":"fallback", "message":"No valid recommendation returned"} to frontend
- Frontend should then:Display a fallback UI message (e.g., "We couldn't generate a recommendation for this scenario. Please review and edit manually or request support.")
- Offer an editable text box for user input or an option to request human review- Log failed prompts, their inputs, and timestamps to a secured prompt log table for review and debugging